Arquitectura desplegada de PoolShop

Nov 9, 2026 · @Juan Cruz

PoolShop corre como un único servicio en Render (plan free, Oregon) que sirve la API Django y el front SvelteKit, conectado a un Postgres 17 también free. Está `live`, pero hay un servicio duplicado que se creó por error y la base expira el 20/11/2026.

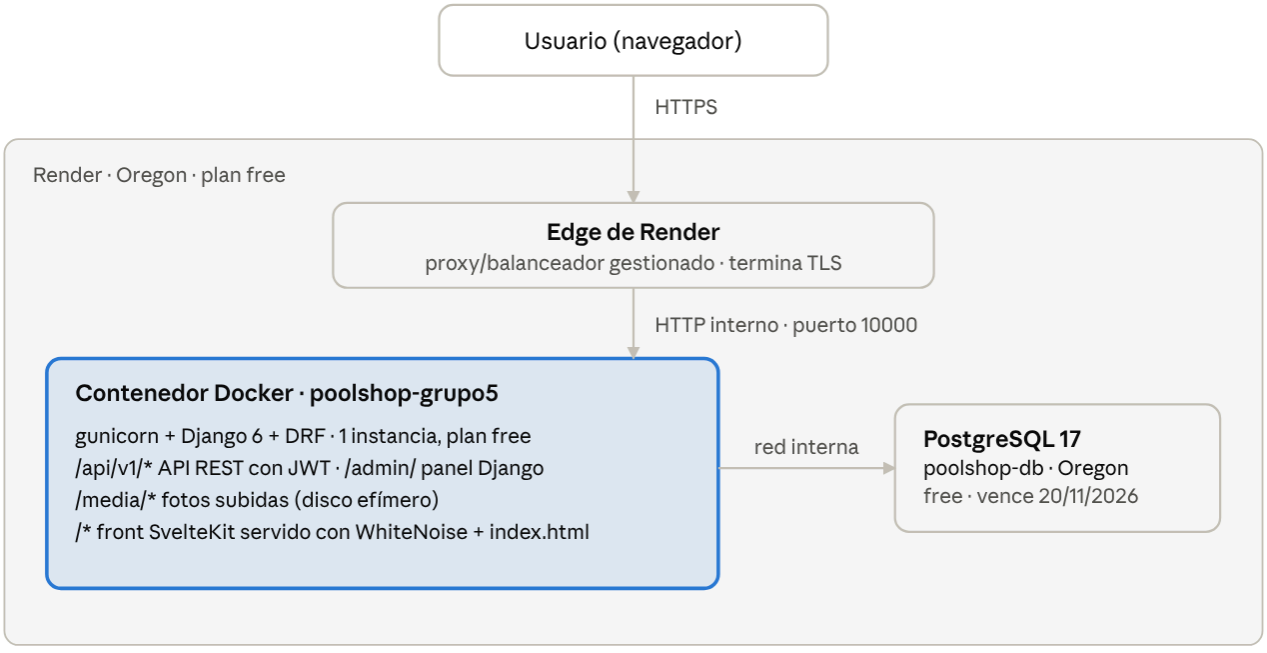
Resumen y hallazgo

En el workspace hay **dos** web services en plan free: `poolshop-grupo5` (el bueno, https://poolshop-grupo5.onrender.com) y `poolshop` (duplicado, https://poolshop-ozy0.onrender.com). El duplicado nació de una primera llamada de creación que devolvió «name already in use» pero igual creó el servicio. Ambos usan el mismo repo, la rama `main` y el mismo Dockerfile, y deberían compartir la misma base de datos (esto último es una deducción: el conector no permite leer las variables de entorno).

El duplicado consume horas del plan free, escribe en la misma base y no tiene `FRONTEND_URL`, así que arma mal los links de los mails de recuperación. Hay que borrarlo desde el dashboard: Settings → Delete Service.

Vista general

Todo el tráfico entra por Render a un único contenedor que habla con Postgres



Sin CDN, caché, colas ni workers propios; el balanceador lo pone Render y no se configura.

arquitectura desplegada · 3 componentes

El usuario llega por HTTPS al edge de Render, que reenvía por HTTP interno al contenedor; el contenedor sirve front, API y media, y consulta Postgres por la red interna de Render.

Red y balanceo

No hay balanceador propio: Render pone un proxy gestionado delante del servicio, que termina TLS y no se configura.

- **Dominio:** solo el subdominio `*.onrender.com`; sin dominio propio ni CDN.
- **HTTPS:** lo gestiona Render. Django lee `X-Forwarded-Proto` y marca las cookies de sesión y CSRF como seguras.
- **Acceso:** `ipAllowList` abierta a `0.0.0.0/0`; sin firewall ni WAF propios.
- **Instancias:** 1, sin autoscaling ni alta disponibilidad.
- **Health check:** no configurado (`healthCheckPath` vacío); Render solo verifica que el puerto responda.

Servicio web

`poolshop-grupo5` es un Web Service de Render con runtime Docker, plan free, región Oregon, que despliega la rama `main` del repo `grupo5-proyectofinal/ProyectoFinal` en cada commit.

Aspecto	Detalle
Build	Dockerfile multi-stage: Node 22 compila el front (`npm ci`, `npm run build`); Python 3.13-slim instala dependencias, copia el front y corre `collectstatic`. Sin caché, así que cada deploy compila de cero.
Arranque	`python manage.py migrate --noinput && gunicorn config.wsgi --bind 0.0.0.0:$PORT --workers 2`. Las migraciones corren en cada arranque.
Ciclo de vida	Se duerme tras ~15 minutos sin tráfico; el primer acceso tarda cerca de un minuto.
Previews de PR	Desactivadas.
Deploys	Uno `live` (commit `5f6f1f5`, merge del PR #2) y uno cancelado, reemplazado al cambiar las variables.

Los logs del primer arranque mostraron migraciones aplicadas, gunicorn 23 con 2 workers y el puerto 10000 abierto.

Aplicación

- **Backend:** Django 6.1.1, DRF 3.18.1, JWT con simplejwt, django-filter y `django-cors-headers`.
- **Frontend:** SvelteKit 2 + Svelte 5 como SPA (`adapter-static` con fallback `index.html`). Llama a la API por ruta relativa `/api/v1`, por lo que no necesita CORS en producción.
- **Estáticos y rutas del SPA:** WhiteNoise sirve los assets de `frontend/build`; una vista de Django devuelve `index.html` para toda ruta que no sea `/api/`, `/admin/`, `/media/` o `/static/`.
- **Media:** `/media/` se sirve con `django.views.static.serve` desde el disco local del contenedor. **Se borra en cada reinicio o redeploy**; afecta a las fotos de pools y de perfiles.
- **Mail:** sin `EMAIL_HOST`, los mails (por ejemplo, recuperación de contraseña) salen por consola y no llegan a nadie.

- **Seguridad:** `DEBUG=false`, cookies de sesión y CSRF seguras, `SECRET_KEY` propia, y `ALLOWED_HOSTS` y `CSRF_TRUSTED_ORIGINS` tomados del hostname que Render inyecta.

Base de datos

PostgreSQL 17 en Render, plan free, región Oregon, estado `available`. **Expira el 20/11/2026**: después Render la suspende y luego la borra.

- **Datos:** base `poolshop_db`, usuario `poolshop_db_user`.
- **Conexión:** Django usa `DATABASE_URL` con la dirección interna de Render y `conn_max_age=600`; sin pool de conexiones externo.
- **Acceso externo:** `ipAllowList` vacío; solo se llega desde dentro de Render. Para operar desde una PC hay que habilitar la IP.
- **Resiliencia:** sin réplicas de lectura, sin alta disponibilidad, sin autoscaling de disco y sin backups en el plan free.

Configuración y lo que no existe

`poolshop-grupo5` tiene cargadas `DEBUG`, `SECRET_KEY`, `DATABASE_URL` y `FRONTEND_URL`. Render agrega `PORT` y `RENDER_EXTERNAL_HOSTNAME` / `RENDER_EXTERNAL_URL`, de donde Django arma `ALLOWED_HOSTS` y `CSRF_TRUSTED_ORIGINS`. Los valores secretos no se reproducen en este documento.

No existen: CDN propio, caché (Redis/Key Value), colas o workers en segundo plano, cron jobs, monitoreo o alertas, backups, entorno de staging, dominio propio, disco persistente ni CI/CD aparte del auto-deploy de Render.

Riesgos y próximos pasos

El más urgente es la base de datos, que vence el `Nov 20, 2026`.

#	Riesgo	Qué hacer
1	La base free expira el 20/11/2026	Migrar a Neon (no vence) o pasar a un plan pago de Render antes de esa fecha
2	Servicio duplicado `poolshop`	Borrarlo en el dashboard: Settings → Delete Service
3	La contraseña de la base quedó escrita en una conversación	Rotarla en el dashboard y actualizar `DATABASE_URL`

#	Riesgo	Qué hacer
4	Fotos subidas efímeras y mails que no salen	Almacenamiento externo (por ejemplo, Cloudinary) y SMTP real con `EMAIL_*`
5	Primer acceso lento y una sola instancia	Aceptarlo en free; un plan pago elimina el cold start

Quedó sin verificar que el front cargue en un navegador: el entorno donde se preparó el despliegue no podía abrir la URL, y tampoco se midió tráfico ni consumo.